

СОГЛАСОВАНО

Руководитель организации-разработчика

Организация-разработчик

_____ И.О. Фамилия

«___» _____ 2026 г.

УТВЕРЖДАЮ

Руководитель организации-заказчика

DV Group

_____ И.О. Фамилия

«___» _____ 2026 г.

**АВТОМАТИЗИРОВАННАЯ СИСТЕМА УПРАВЛЕНИЯ ПРОДАЖАМИ
ОБЪЕКТОВ НЕДВИЖИМОСТИ
«CRM Недвижимость»**

ОПИСАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

XXXX.XXXXXXX.001.ПА

Листов 12

Ташкент 2026

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ ТЕРМИНОВ И СОКРАЩЕНИЙ.....	3
1 Структура программного обеспечения.....	5
2 Функции частей программного обеспечения.....	6
2.1 Серверная часть	6
2.2 Клиентская часть	7
2.3 Обратный прокси и развёртывание.....	7
3 Методы и средства разработки программного обеспечения.....	8
4 Операционная система	9
5 Средства, расширяющие возможности операционной системы.....	10
5.1 Состав общесистемного ПО и библиотек	10
5.2 Требования к ПО при развитии	11
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ	12

ПЕРЕЧЕНЬ ТЕРМИНОВ И СОКРАЩЕНИЙ

Таблица 1 – Сокращения

Сокращение	Расшифровка
АС	Автоматизированная система
БД	База данных
ВК	Внешний ключ таблицы базы данных
ЕСПД	Единая система программной документации
ЖК	Жилой комплекс
ИО	Информационное обеспечение
ОС	Операционная система
ПИНФЛ	Персональный идентификационный номер физического лица
ПК	Первичный ключ таблицы базы данных
ПО	Программное обеспечение
РОП	Руководитель отдела продаж
СУБД	Система управления базами данных
ТЗ	Техническое задание
УТП	Уникальное торговое предложение
API	Программный интерфейс приложения (Application Programming Interface)
CRM	Система управления взаимоотношениями с клиентами (Customer Relationship Management)
HTTPS	Протокол передачи гипертекста с шифрованием
JWT	Формат токена доступа (JSON Web Token)
REST	Архитектурный стиль программного интерфейса поверх HTTP
UZS, USD, EUR	Коды валют: узбекский сум, доллар США, евро

Таблица 2 – Термины

Термин	Определение
Бронь	Сделка в начальном статусе: объект закреплён за клиентом до даты окончания брони и выведен из продажи
График платежей	Разбивка стоимости по договору на платежи с плановыми датами; сумма платежей равна стоимости по договору
Заявка	Зарегистрированное обращение клиента с его интересом к приобретению недвижимости; основная единица воронки продаж
Клиент	Физическое лицо, с которым работает компания; карточка клиента принадлежит одной компании
Компания	Юридическое лицо, работающее в системе; данные и справочники разных компаний изолированы друг от друга
Мягкое удаление	Пометка записи удалённой без физического удаления: запись скрывается из списков и отчётов и может быть восстановлена
Область видимости	Граница данных, доступных пользователю по разрешению: свои записи, отдел, компания или вся система
Объект недвижимости, объект	Квартира, коммерческое помещение, парковочное место, кладовая или коттедж в составе дома
Партнёр	Внешняя система (сайт, бот, агентство), обращающаяся к публичному API по API-ключу
Разрешение	Сочетание действия, ресурса и области видимости, например «просмотр клиентов своего отдела»
Расторжение	Прекращение сделки, по которой прошли платежи или есть подпись клиента; требует документа-основания и влечёт возврат денег
Роль	Именованный набор разрешений, назначаемый пользователям
Сделка	Договорные отношения с клиентом по одному объекту: от брони до подписанного договора или отмены
Справочник	Перечень значений для выбора в формах (статусы заявок, причины отказа, типы платежей и т.п.)
Шахматка	Табличное представление объектов дома: подъезды по столбцам, этажи по строкам, статус — цветом ячейки

1 Структура программного обеспечения

Программное обеспечение системы состоит из общесистемного и специального ПО.

Общесистемное ПО — операционная система сервера, контейнерная платформа, СУБД, хранилище в оперативной памяти, обратный прокси, интерпретаторы и сторонние библиотеки. Оно распространяется его правообладателями и в составе системы не изменяется.

Специальное ПО разработано для системы и включает:

- серверную часть — веб-приложение на Python (фреймворк Django) с программным интерфейсом в стиле REST, командами управления и миграциями БД (каталог `backend`);
- клиентскую часть — одностраничное веб-приложение на TypeScript (библиотека React), исполняемое в браузере (каталог `frontend-new`);
- конфигурацию обратного прокси (каталог `nginx`) и описание развёртывания (`docker-compose.yml`, файлы `Dockerfile`);
- инструменты сборки документации (каталог `docs`).

Исходные тексты специального ПО хранятся в репозитории Git; опись приведена в документе «Текст программы».

2 Функции частей программного обеспечения

2.1 Серверная часть

Серверная часть разделена на приложения Django; каждое реализует модели данных, сериализаторы (проверка входных данных и формирование ответов), представления программного интерфейса, фильтры и маршруты своей предметной области.

Таблица 3 – Модули серверной части

Модуль	Функции
real_estate_project	Настройки системы, корневые маршруты, проверка работоспособности (/api/health/), выдача файлов с проверкой подписи (media_access.py), постраничная выдача (pagination.py), публикация описания API (/api/schema/)
permissions	Компании, отделы, профили пользователей, разрешения и роли; проверка разрешений и отбор данных по области видимости (backends.py, permissions.py); общие правила справочников компании (reference_scope.py); вход, выход, обновление токенов, cookie токена обновления, восстановление пароля, блокировка при подборе пароля (auth_views.py, token_serializers.py, token_cookies.py); ограничение частоты запросов (throttles.py); API-ключи партнёров и их шифрование (crypto.py); журнал изменений прав
apps.realty	Проекты, дома, объекты, планировки, галереи, типы домов, скидки; загрузка объектов из XLSX и выдача шаблона; массовая загрузка изображений планировок
apps.crm	Клиенты и их телефоны (phones.py — приведение номеров к единому виду), файлы клиентов, заявки, встречи, справочники заявок; мягкое удаление и восстановление; сводные таблицы; дашборд; публичный API партнёров
apps.deals	Сделки: бронь с блокировкой объекта, условия, доступные скидки, отмена и расторжение, автоматическая смена статусов сделки, объекта и заявки
apps.finances	Платежи, график платежей сделки, отметка оплаты и возврата, освобождение объекта после возвратов, сводная таблица, справочники типов платежей и счетов получателей; валюты и курсы (currency.py — загрузка курсов ЦБ, курс на дату, пересчёт сумм; currency_views.py — настройки валют компании и курсы; installment_views.py — условия рассрочки)
apps.documents	Шаблоны договоров и формирование документа DOCX по сделке (библиотека docxtempl, изолированная среда шаблонизатора)
apps.tasks	Задачи, подзадачи, комментарии, журнал задач, представления канбан и календарь, статистика
apps.reports	Планы продаж по проектам и сотрудникам (шаблоны и загрузка XLSX), отчёты «план-факт»

Таблица 4 – Команды управления серверной частью

Команда	Назначение
init_permissions	Создание разрешений по всем сочетаниям действий, ресурсов и областей; типовых ролей — при первичной установке или с ключом --with-roles; с ключом --sync-roles — обновление состава прав существующих типовых ролей
refresh_overdue_payments	Перевод неоплаченных платежей с прошедшим сроком в статус «Просрочен» (регламентное задание)
expire_bookings	Отмена броней с истёкшей датой окончания и возврат объектов в продажу (регламентное задание)

Команда	Назначение
close_stale_meetings	Закрытие встреч, просроченных более чем на 7 дней (регламентное задание)
show_access <логин>	Вывод флагов учётной записи, ролей и источника каждого права на справочники — для разбора обращений о доступе
grant_settings_access	Выдача права «Просмотр» на ресурс «Настройки» ролям, которые ведут настроенные справочники (при переходе на отдельное право доступа к разделу)
check_company_isolation	Показ записей справочников и данных без компании (общесистемных); при необходимости — закрепление их за указанной компанией
find_duplicate_units	Поиск повторяющихся номеров объектов в доме; разбор дублей обязателен до применения миграции <code>realty.0013</code> , делающей номер уникальным в доме
setup_admin_permissions	Настройка полных прав для администратора
reset_passwords	Сброс паролей учётных записей (для тестовых сред)
create_test_users	Создание тестовых пользователей (для тестовых сред)

2.2 Клиентская часть

Клиентская часть собирается инструментом Vite в набор статических файлов и выполняется в браузере. Структура исходных текстов приведена в таблице 5.

Таблица 5 – Структура клиентской части

Каталог <code>src</code>	Содержимое
<code>api</code>	Функции обращения к программному интерфейсу; клиент HTTP с подстановкой токена доступа и его обновлением; хранение токена доступа в памяти (<code>tokenStore.ts</code>)
<code>pages</code>	Страницы разделов: дашборд, клиенты, заявки, встречи, задачи, сделки, проекты, дома, финансы, отчёты, скидки, настройки, страницы входа и восстановления пароля
<code>components</code>	Компоненты разделов (формы, карточки, канбан, шахматка, график платежей, генерация документов) и общие компоненты (таблица с локализацией, мобильный список карточек, поля дат)
<code>layouts</code>	Каркас приложения: меню, верхняя панель, переключатели языка и темы
<code>store</code>	Состояние приложения: сведения о пользователе и сессии, тема оформления
<code>utils</code>	Проверка разрешений в интерфейсе, состав вкладок настроек, перевод кодов классификаторов, разбор ошибок сервера
<code>i18n</code>	Настройка локализации и файлы переводов <code>ru.json</code> , <code>en.json</code> , <code>uz.json</code>
<code>hooks, styles, assets</code>	Общие хуки, стили, изображения

2.3 Обратный прокси и развёртывание

Конфигурация обратного прокси (`nginx/nginx.conf`, `nginx/conf.d/*.conf`) задаёт маршрутизацию запросов, зоны ограничения частоты запросов, заголовки безопасности, прямую отдачу общедоступных изображений и внутреннее расположение `/protected-media/` для отдачи защищённых файлов. Для работы по HTTPS используется шаблон `default-https.conf.template`. Файл `docker-compose.yml` описывает контейнеры, сети, тома и параметры среды.

3 Методы и средства разработки программного обеспечения

Таблица 6 – Средства разработки

Назначение	Средства
Языки программирования	Python 3.12, TypeScript 5.8
Серверный фреймворк	Django 5.2, Django REST framework 3.16
Клиентская библиотека	React 19, MUI 7, MUI X 8
Сборка клиентской части	Vite 7, Node.js 20
Проверка кода	ESLint, проверка типов TypeScript (tsc)
Автоматические тесты	Тестовый каркас Django (manage.py test)
Описание API	drf-spectacular (OpenAPI 3)
Контроль версий	Git
Контейнеризация	Docker, Docker Compose

Проектирование серверной части выполнено по принципу «тонкие представления — правила в моделях и сериализаторах»: проверка входных данных и бизнес-правил сосредоточена в сериализаторах и методах представлений, составные изменения выполняются в транзакциях. Доступ к данным проверяется на сервере независимо от интерфейса.

4 Операционная система

Серверная часть, СУБД, хранилище Redis и обратный прокси работают в контейнерах на базе Linux (Debian для образа серверной части `python:3.12-slim`, Alpine Linux для остальных образов). Сервер размещения — любая ОС семейства Linux с Docker Engine и Docker Compose (рекомендуется Ubuntu Server LTS или Debian). Для разработки допускаются Windows, macOS и Linux.

На рабочих местах пользователей требуется только веб-браузер: актуальные версии Google Chrome, Microsoft Edge, Mozilla Firefox, Safari, в том числе мобильные.

5 Средства, расширяющие возможности операционной системы

5.1 Состав общесистемного ПО и библиотек

Таблица 7 – Общесистемное ПО

Компонент	Версия	Лицензия	Назначение
Docker Engine, Docker Compose	актуальная	Apache 2.0	Контейнерная платформа
PostgreSQL	16	PostgreSQL License	СУБД
Redis	7	BSD-3 (до 7.2), RSALv2/SSPLv1 (с 7.4)	Кэш, лимиты, блокировки входа
nginx	актуальная стабильная	BSD-2	Обратный прокси, отдача файлов
Python	3.12	PSF License	Интерпретатор серверной части
gunicorn	актуальная	MIT	Сервер приложений WSGI
Node.js	20 (только сборка)	MIT	Сборка клиентской части
postgres-backup-local	16	MIT	Резервное копирование БД

Таблица 8 – Библиотеки серверной части

Библиотека	Версия	Лицензия	Назначение
Django	5.2.6	BSD-3	Веб-фреймворк, ORM, миграции
djangorestframework	3.16.1	BSD-3	Программный интерфейс REST
djangorestframework-simplejwt	5.5.1	MIT	Токены JWT, чёрный список
django-filter	25.1	BSD-3	Фильтры списков
django-cors-headers	4.9.0	MIT	Политика CORS
drf-spectacular	0.28.0	BSD-3	Описание API OpenAPI 3
psycopg2-binary	2.9.11	LGPL-3.0	Драйвер PostgreSQL
redis	5.2.1	MIT	Клиент Redis
cryptography	44.0.0	Apache 2.0 / BSD-3	Шифрование API-ключей (Fernet)
PyJWT	2.10.1	MIT	Работа с JWT
python-decouple	3.8	MIT	Чтение параметров среды
openpyxl	3.1.5	MIT	Чтение и запись XLSX
pandas	2.3.3	BSD-3	Обработка табличных данных при загрузке
numpy	2.3.3	BSD-3	Зависимость pandas
python-docx	1.2.0	MIT	Работа с DOCX
docxtpl	0.20.1	LGPL-2.1	Подстановка данных в шаблоны DOCX
docxcompose	1.4.0	MIT	Зависимость docxtpl
Jinja2	3.1.6	BSD-3	Шаблонизатор (изолированная среда)
lxml	6.0.2	BSD-3	Разбор XML документов
pillow	11.3.0	MIT-CMU	Обработка изображений
babel, python-dateutil, pytz, tzdata	по requirements.txt	BSD / Apache 2.0	Даты, часовые пояса, локали

Таблица 9 – Библиотеки клиентской части

Библиотека	Версия	Лицензия	Назначение
react, react-dom	19	MIT	Построение интерфейса

Библиотека	Версия	Лицензия	Назначение
react-router-dom	7	MIT	Маршрутизация страниц
@mui/material, @mui/icons-material, @mui/lab	7	MIT	Компоненты интерфейса
@mui/x-data-grid, x-date-pickers, x-charts, x-tree-view	8	MIT	Таблицы, выбор дат, графики, деревья
@tanstack/react-query	5	MIT	Загрузка и кэширование данных
axios	1	MIT	Запросы HTTP
react-hook-form	7	MIT	Формы и проверка ввода
zustand, @reduxjs/toolkit, react-redux, redux-persist	5, 2, 9, 6	MIT	Состояние приложения
i18next, react-i18next, i18next-browser-languagedetector	25, 16, 8	MIT	Локализация
date-fns	4	MIT	Работа с датами
typescript	5.8	Apache 2.0	Язык и проверка типов (сборка)
vite, @vitejs/plugin-react	7, 5	MIT	Сборка (сборка)
eslint и расширения	9	MIT	Проверка кода (разработка)

Примечание – Точные версии библиотек фиксируются файлами backend/requirements.txt и frontend-new/package-lock.json. Сведения о лицензиях приведены по данным правообладателей для указанных версий и подлежат проверке при обновлении.

5.2 Требования к ПО при развитии

При обновлении общесистемного ПО и библиотек необходимо:

- а) обновлять версии в requirements.txt и package.json с фиксацией в package-lock.json;
- б) выполнять автоматические тесты серверной части, проверку типов и сборку клиентской части;
- в) проверять лицензию новой версии компонента;
- г) при обновлении PostgreSQL на новую основную версию выполнять перенос данных через резервную копию по документу «Руководство системного программиста».

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]